Code Classroom : **gaklp6rc**

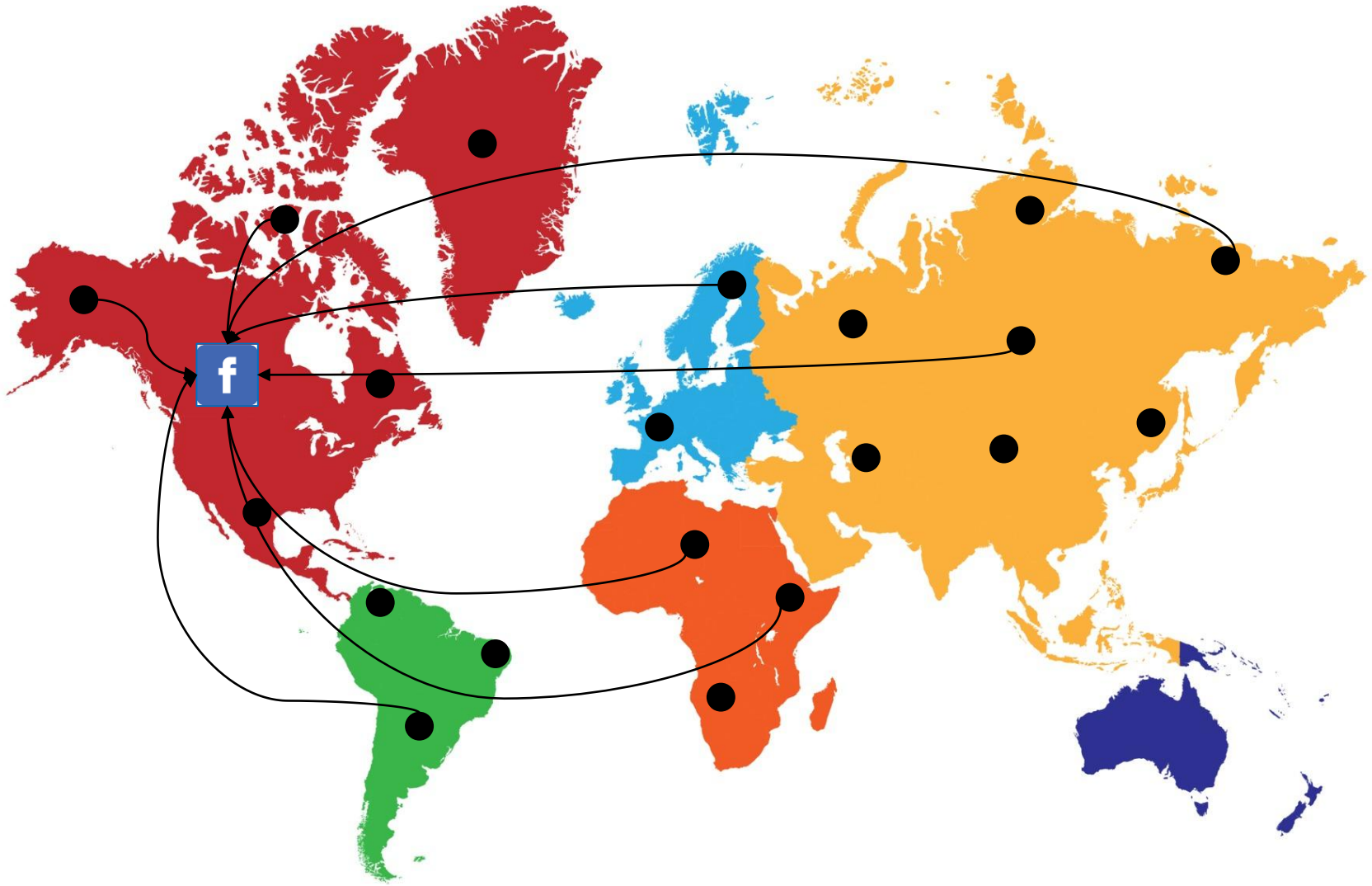
Développement d'applications réparties

Dr. Mouna KARAJA

Niveau : 3ème année GLSI

Année universitaire : 2025-2026

Contexte et enjeux



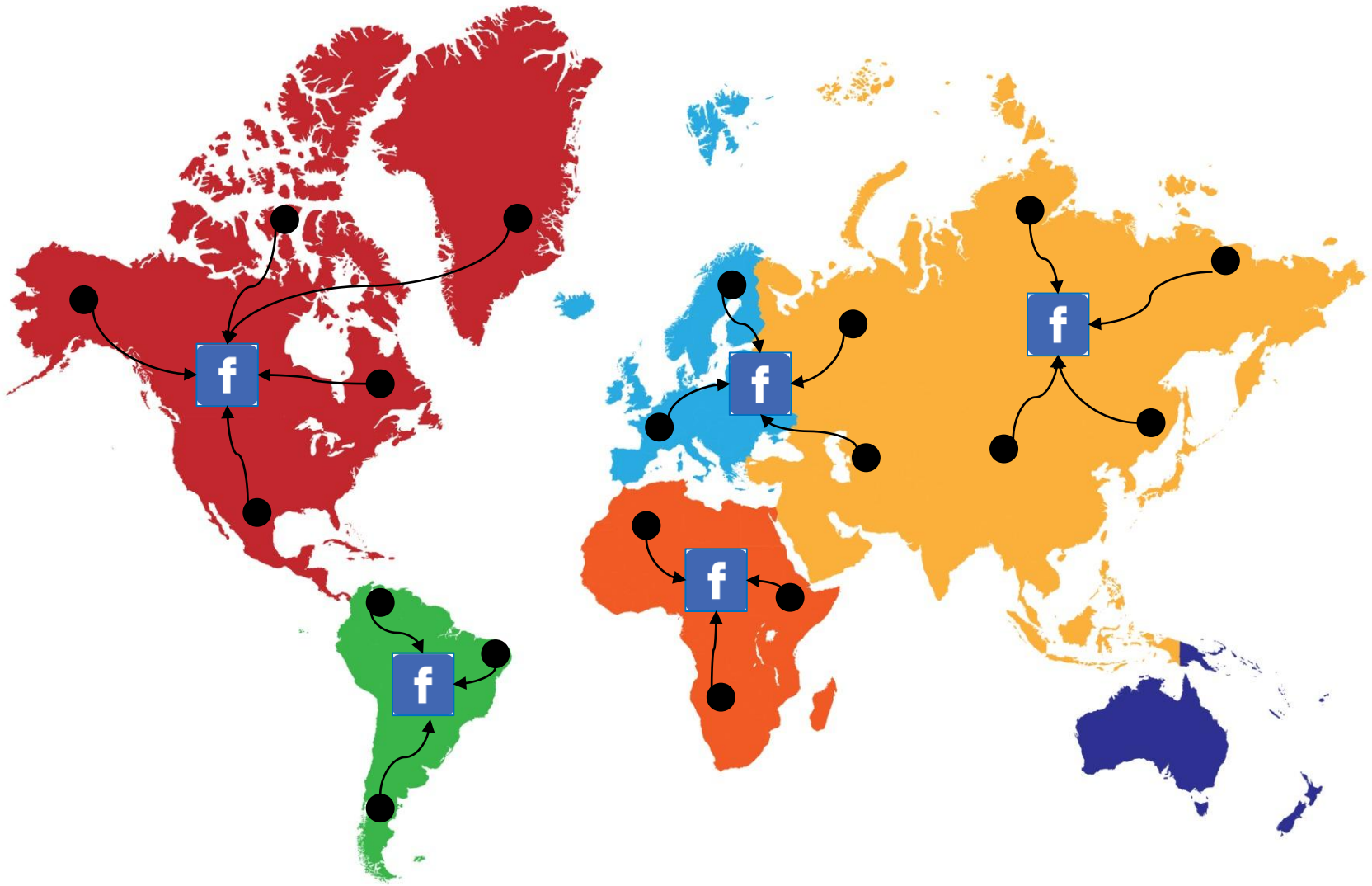
Le cas naïf (un seul serveur dans le monde) :

- Tous les utilisateurs, qu'ils soient en Tunisie, en Europe ou en Asie, doivent se connecter à un seul serveur situé en Amérique.

Résultat :

- ✓ lenteur,
- ✓ surcharge,
- ✓ risque de panne totale si ce serveur tombe.

Contexte et enjeux



Le cas réel (serveurs répartis dans le monde) :

- Plusieurs serveurs répartis géographiquement (Amérique, Europe, Asie, Afrique…).
- Chaque utilisateur se connecte au serveur le plus proche.

Avantages :

- ✓ rapidité,
- ✓ meilleure disponibilité,
- ✓ tolérance aux pannes,
- ✓ qualité de service.

Contexte et enjeux

Le monde d'aujourd'hui est **connecté**. Les applications que nous utilisons tous les jours reposent toutes sur un principe fondamental : elles sont **réparties**.

➡ Cela signifie qu'elles ne s'exécutent pas sur une seule machine, mais sur un ensemble d'ordinateurs, serveurs ou dispositifs qui coopèrent pour offrir un service unique, rapide, fiable et accessible partout dans le monde.

Exemple : Facebook

- Quand un utilisateur ouvre Facebook, il accède à une interface sur son smartphone ou son navigateur.
- Les données de son profil (amis, publications, photos) sont stockées dans différents serveurs répartis dans le monde.
- Quand il publie une photo, cette photo est envoyée à un serveur de stockage, répliquée dans plusieurs centres de données (pour la sauvegarde et la rapidité d'accès), et rendue disponible à ses amis à travers d'autres serveurs.
- Quand il like ou commente, l'information passe par d'autres serveurs qui mettent à jour la base de données et notifient instantanément ses amis.

→ Pourtant, pour l'utilisateur, Facebook semble être une seule application fluide.

Contexte et enjeux

applications réparties = applications distribuées

Une **application répartie** est une application dont les composants logiciels s'exécutent sur plusieurs ordinateurs (ou nœuds) reliés par un réseau, et qui coopèrent pour fournir un service unique et cohérent à l'utilisateur final.

- ✓ Les composants peuvent être des programmes, des bases de données, des services web, etc.
- ✓ Ces composants sont géographiquement dispersés, mais ils travaillent comme un tout.
- ✓ L'utilisateur ne perçoit pas cette répartition : il a l'impression d'interagir avec une seule application.









Contexte et enjeux

Pourquoi cette matière est importante ?

D'un point de vue académique (GLSI)

Comprendre les applications réparties est essentiel pour :

- ★ Développement d'applications web et mobiles évolutives.
- ★ Conception d'architectures logicielles adaptées aux systèmes distribués.
- ★ Proposer et réaliser un sujet de PFE moderne et intéressant.

D'un point de vue professionnel

Les entreprises recherchent des diplômés en informatique capables de :

- ★ Gérer la communication entre machines et services à grande échelle.
- ★ Concevoir des systèmes distribués fiables, performants et maintenables.
- ★ Garantir la sécurité et la tolérance aux pannes dans des environnements critiques (banques, télécoms, etc.).

Contexte et enjeux

Objectifs de la matière

Comprendre les concepts et mécanismes permettant à des ordinateurs dispersés de coopérer.

Maîtriser les technologies et protocoles de communication et de synchronisation.

Découvrir les modèles architecturaux qui dominent le monde logiciel actuel.

Apprendre les stratégies de tolérance aux pannes et de sécurité pour garantir la disponibilité des services.

Plan de la matière

1. Introduction aux systèmes et applications réparties
2. Communication dans les applications réparties
3. Architectures des applications réparties
4. Tolérance aux pannes et sécurité

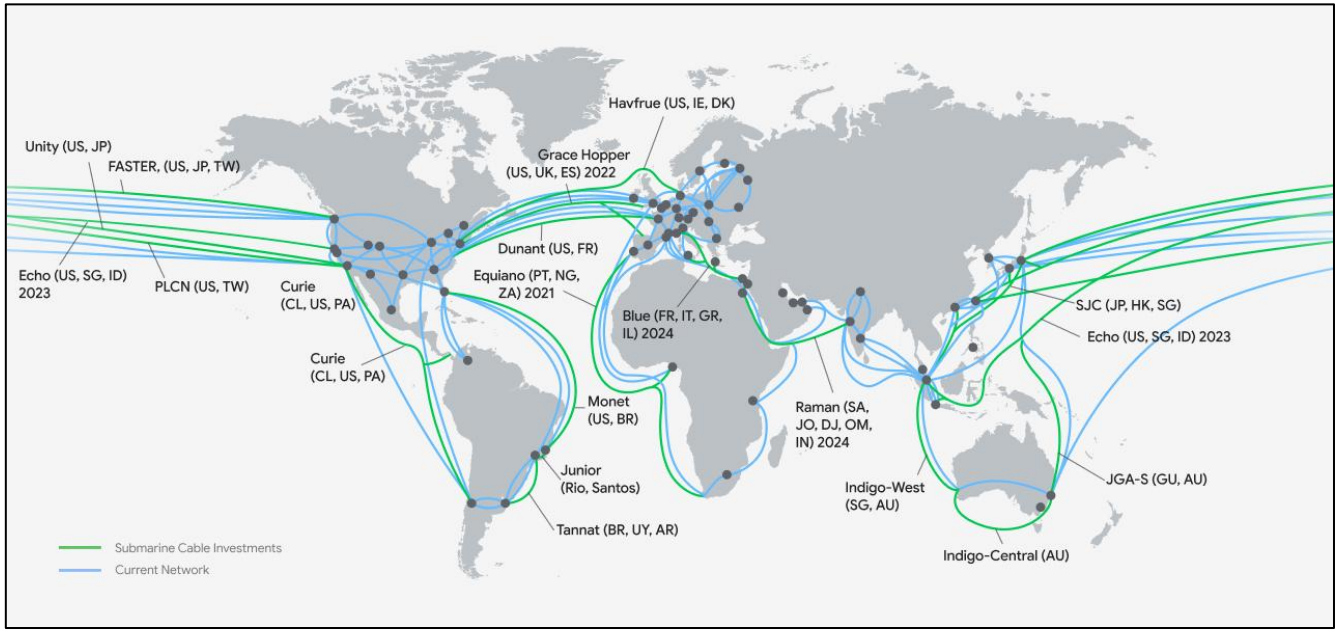
Chapitre 1

Introduction aux systèmes et applications réparties

Qu’est ce qu’un système réparti ?

Un **système réparti** est un ensemble de machines autonomes reliées par un réseau, qui coopèrent pour offrir l’illusion d’un système unique et cohérent.

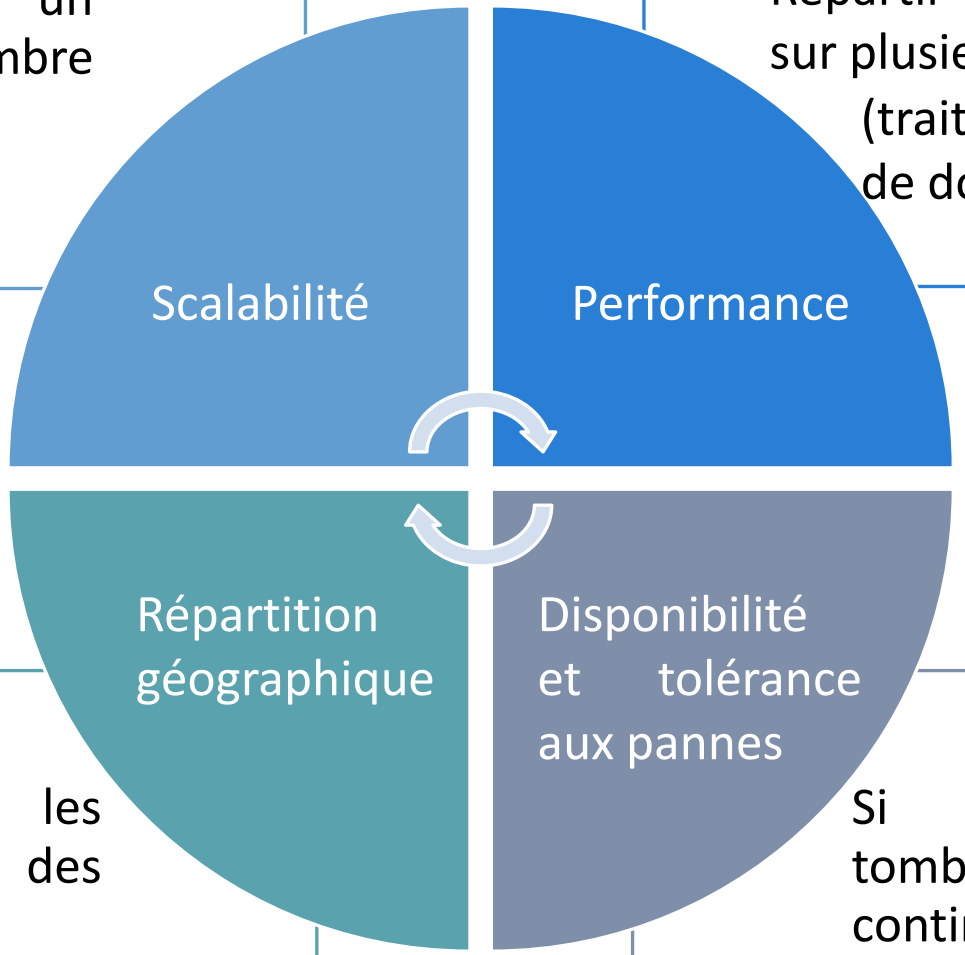
Exemple : un cluster de serveurs qui hébergent un moteur de recherche comme Google.



Caractéristiques :

- ✓ Les ressources (processeurs, mémoire, données) sont distribuées.
- ✓ Les utilisateurs interagissent comme s’ils accédaient à une seule machine centrale.
- ✓ Les communications passent obligatoirement par un réseau.

Pourquoi les systèmes répartis ?



Supporter un grand nombre d'utilisateurs.

Répartir le traitement sur plusieurs machines (traitement massif de données).

Rapprocher les ressources des utilisateurs.

Si un serveur tombe, d'autres continuent à fournir le service.

Programmation classique vs. répartie

Critère	Programmation classique	Programmation répartie
Lieu d'exécution	Sur une seule machine (serveur ou poste local)	Répartie sur plusieurs machines connectées en réseau
Partage des ressources	Ressources limitées à la machine locale	Ressources partagées entre plusieurs nœuds (processeurs, mémoire, stockage)
Communication	Appels locaux, simples et rapides	Communication via réseau (messages, protocoles, latences)
Tolérance aux pannes	Une panne = arrêt du système	Conçue pour être tolérante aux pannes et redondante
Evolutivité	Limitée par la capacité matérielle de la machine	Haute évolutivité (ajout de nouveaux nœuds possible)
Complexité de développement	Plus simple (modèle linéaire, centralisé)	Plus complexe (synchronisation, concurrence, cohérence des données)
Exemple	Application de bureau (MS Word, Excel)	Applications web distribuées (réseaux sociaux, systèmes bancaires, e-commerce)

Système réparti vs. Application répartie

Système réparti

Un système réparti est un ensemble d'entités autonomes (ordinateurs, processeurs, processus) reliées par un réseau, qui coopèrent pour accomplir une tâche commune.

- ✓ Ces entités sont physiquement distinctes, mais apparaissent comme un tout cohérent.
- ✓ Le système réparti gère la communication, la coordination et le partage des ressources entre ses composants.

Exemple : Internet

➡ C'est l'infrastructure matérielle et logicielle (réseau, serveurs, protocoles, …).

Application répartie

Une application répartie est un ensemble d'entités logicielles (programmes, modules, services) exécutées sur des environnements différents et interconnectés via un réseau.

- ✓ Elle exploite les services offerts par le système réparti pour fonctionner.
- ✓ C'est donc un logiciel distribué qui utilise l'infrastructure sous-jacente.

Exemple : WhatsApp, Netflix, Google Docs.

➡ C'est un logiciel construit au-dessus de cette infrastructure pour offrir un service aux utilisateurs.

Les applications réparties

Une application répartie n'est pas un bloc monolithique :

- Elle est composée de services (authentification, messagerie, paiement, etc.).
- Chaque service peut être hébergé sur une machine différente, mais l'ensemble coopère pour offrir une application cohérente.
- Ces services doivent être accessibles via des interfaces bien définies. Ces interfaces peuvent prendre différentes formes, telles que des API ou des protocoles de communication.

Dans les applications réparties, on distingue souvent :

- Les services représentent les fonctionnalités offertes (par exemple : authentification, messagerie).
- Les interfaces représentent le moyen par lequel ces services sont accessibles.

  C'est comme un contrat qui définit :
  - Quelles opérations on peut demander au service (ex. : s'authentifier, envoyer un message, publier un contenu).
  - La manière dont ces opérations doivent être appelées (ex. : via un protocole HTTP, une API REST, un message JSON).

Les applications réparties

Dans un système réparti, un service peut être offert par plusieurs serveurs et consommé par plusieurs applications.

Exemple : Facebook propose plusieurs services répartis : le service d'authentification, le service de messagerie instantanée, le service de publication et partage de contenu, le service de recherche, etc.

→ Ces services, bien que physiquement répartis, apparaissent à l'utilisateur comme une seule application.

❖ Le service d'authentification permet à l'utilisateur de se connecter.
❖ Son interface définit les règles :
- ✓ Quelles données envoyer **?** (login, mot de passe)
- ✓ Comment les envoyer **?** (par un formulaire Web, ou via une API pour une application mobile)
- ✓ Ce que le service renvoie **?** (succès ou erreur)

Ainsi, même si les services sont physiquement répartis (hébergés sur différents serveurs), l'utilisateur y accède toujours de la même manière grâce à leurs interfaces bien définies.

Impact sur le génie logiciel

Les systèmes répartis modifient profondément la manière de concevoir et développer des logiciels :

Nouvelles contraintes	• Latence, panne réseau, sécurité des communications
Architectures logicielles	• Apparition des modèles client-serveur, microservices, services web.
Méthodologies	• Nécessité de penser en termes de scalabilité, tolérance aux pannes, réplication de données.
Maintenance et évolutivité	• Conception modulaire pour faire évoluer l'application sans arrêter tout le système.

Le génie logiciel appliqué aux systèmes répartis exige des approches spécifiques (patrons d'architecture, middlewares, protocoles de communication, etc.).

Historique et évolution

1960-1970

Premiers réseaux d'ordinateurs, début de l'idée de partage des ressources.

Années 1980

Emergence des systèmes répartis universitaires et industriels (SUN RPC, CORBA, systèmes client-serveur).

Années 1990

Démocratisation d'Internet → explosion des applications web réparties (serveurs web, bases de données distribuées).

Années 2000

Apparition du cloud computing (Amazon Web Services, Google Cloud), systèmes massivement parallèles.

Années 2010

Montée du big data, de l'IoT, du fog/edge computing, microservices et conteneurs (Docker, Kubernetes).

Aujourd'hui

Intelligence artificielle distribuée, blockchain, edge AI et applications massivement interconnectées.